

- a. I used ChatGPT and Google Gemini.
- b. I accessed ChatGPT using the MacOS app. Google Gemini was accessed via `gemini.google.com` in the browser.
- c. The prompt

Create a C++ program using objects, not functions, that prioritizes emails for a busy company CEO. Implement a MaxHeap from scratch using a list-based implementation and use it as a priority queue. Do not use any pre-existing heap modules.

The program should read commands from a test file. An EMAIL command has the format:
EMAIL ,,

Sender categories, from highest to lowest priority, are: Boss, Subordinate, Peer, ImportantPerson, and OtherPerson. Emails with a higher-priority sender category must be read before lower-priority categories. If multiple emails have the same sender category, the newest email must be read first. Dates are in MM-DD-YYYY format. Subject lines may contain spaces but will not contain commas.

Support these commands:

NEXT — Display the highest-priority unread email in the format:

Sender:

Subject:

Date:

READ — Remove the highest-priority unread email from the queue.

COUNT — Display the current number of unread emails.

The program must correctly handle NEXT or READ when there are no emails, multiple NEXT commands in a row without READ (which should display the same email), and multiple READ commands in a row without NEXT (each READ should remove the current highest-priority email).

Use object-oriented C++ and create the MaxHeap implementation from scratch. The program should compile and run in a Linux environment using standard C++. Include clear comments explaining the code.

- d. ChatGPT code

```
#include <iostream>
```

```
#include <fstream>

#include <sstream>

#include <string>

#include <list>

#include <stdexcept>

using namespace std;

/*

 * Email represents one unread email.

 *

 * Higher senderPriority means the email has higher priority.

 * If two emails have the same sender priority, the newer date

 * has higher priority.

 */

class Email

{

private:

    string sender;

    string subject;

    string date;

    int senderPriority;

    int dateValue;

    // Convert MM-DD-YYYY into YYYYMMDD.
```

```
// This makes chronological comparison easy.
```

```
int convertDate(const string& dateString) const
```

```
{
```

```
    int month, day, year;
```

```
    char dash1, dash2;
```

```
    stringstream ss(dateString);
```

```
    ss >> month >> dash1 >> day >> dash2 >> year;
```

```
    return year * 10000 + month * 100 + day;
```

```
}
```

```
// Assign numeric priorities to sender categories.
```

```
int getSenderPriority(const string& senderName) const
```

```
{
```

```
    if (senderName == "Boss")
```

```
        return 5;
```

```
    else if (senderName == "Subordinate")
```

```
        return 4;
```

```
    else if (senderName == "Peer")
```

```
        return 3;
```

```
    else if (senderName == "ImportantPerson")
```

```
        return 2;
```

```
    else
```

```
        return 1; // OtherPerson
```

```
}
```

```
public:
```

```
    Email()
```

```
{
```

```
    senderPriority = 0;
```

```
    dateValue = 0;
```

```
}
```

```
    Email(const string& senderName,
```

```
          const string& subjectLine,
```

```
          const string& emailDate)
```

```
{
```

```
    sender = senderName;
```

```
    subject = subjectLine;
```

```
    date = emailDate;
```

```
    senderPriority = getSenderPriority(sender);
```

```
    dateValue = convertDate(date);
```

```
}
```

```
/*
```

```
 * Return true if this email should have higher priority
```

```
 * than another email.
```

```
*/
```

```
bool operator>(const Email& other) const
```

```

{
    // Sender category is the primary priority.

    if (senderPriority != other.senderPriority)

        return senderPriority > other.senderPriority;

    // Within the same category, newest email comes first.

    return dateValue > other.dateValue;
}

void display() const
{
    cout << "Sender: " << sender << endl;

    cout << "Subject: " << subject << endl;

    cout << "Date: " << date << endl;
}

};

/*
* MaxHeap
*
* This class implements a MaxHeap from scratch.
*
* A std::list is used as the underlying storage. No heap or
* priority_queue from the standard library is used.
*

```

```
* Although a list does not provide random-access indexing,  
* getIterator() allows the heap to access the logical position  
* of an element in the list.
```

```
*/
```

```
class MaxHeap
```

```
{
```

```
private:
```

```
    list<Email> heap;
```

```
    /*
```

```
    * Return an iterator to the item at the specified logical
```

```
    * index in the list.
```

```
    */
```

```
    list<Email>::iterator getIterator(int index)
```

```
{
```

```
    list<Email>::iterator it = heap.begin();
```

```
    for (int i = 0; i < index; i++)
```

```
        ++it;
```

```
    return it;
```

```
}
```

```
    /*
```

```
    * Restore the heap property after a new email is inserted.
```

```
    */
```

```

void heapifyUp(int index)
{
    while (index > 0)
    {
        int parentIndex = (index - 1) / 2;

        list<Email>::iterator current = getIterator(index);
        list<Email>::iterator parent = getIterator(parentIndex);

        if (*current > *parent)
        {
            // Swap the Email objects stored at these positions.

            Email temp = *current;

            *current = *parent;

            *parent = temp;

            index = parentIndex;
        }
        else
        {
            break;
        }
    }
}

/*

```

* Restore the heap property after the maximum item

* has been removed.

*/

void heapifyDown(int index)

{

int heapSize = static_cast<int>(heap.size());

while (true)

{

int leftChild = 2 * index + 1;

int rightChild = 2 * index + 2;

int largest = index;

// Compare with left child.

if (leftChild < heapSize)

{

list<Email>::iterator left = getIterator(leftChild);

list<Email>::iterator largestIt = getIterator(largest);

if (*left > *largestIt)

largest = leftChild;

}

// Compare with right child.

if (rightChild < heapSize)

{

```

        list<Email>::iterator right = getIterator(rightChild);

        list<Email>::iterator largestIt = getIterator(largest);

        if (*right > *largestIt)

            largest = rightChild;

    }

    // Heap property is satisfied.

    if (largest == index)

        break;

    list<Email>::iterator current = getIterator(index);

    list<Email>::iterator largestIt = getIterator(largest);

    Email temp = *current;

    *current = *largestIt;

    *largestIt = temp;

    index = largest;

}

}

public:

    MaxHeap()

    {

    }

    /*

    * Add a new email to the heap.

```

```

*/

void insert(const Email& email)
{
    heap.push_back(email);

    int newIndex = static_cast<int>(heap.size()) - 1;

    heapifyUp(newIndex);
}

/*
 * Return the highest-priority email without removing it.
 */

const Email& getMax() const
{
    if (heap.empty())

        throw runtime_error("Heap is empty.");

    return heap.front();
}

/*
 * Remove the highest-priority email.
 */

void removeMax()
{
    if (heap.empty())

```

```

        return;

// Only one item exists.

if (heap.size() == 1)
{
    heap.pop_front();

    return;
}

/*
 * Move the last item into the root position and
 * delete the old last item.
 */

heap.front() = heap.back();

heap.pop_back();

// Restore MaxHeap ordering.

heapifyDown(0);
}

bool empty() const
{
    return heap.empty();
}

int size() const
{

```

```

        return static_cast<int>(heap.size());

    }

};

/*

* EmailPriorityQueue provides the priority-queue operations
* needed by the CEO's email system.

*/

class EmailPriorityQueue

{

private:

    MaxHeap emailHeap;

public:

    void addEmail(const Email& email)

    {

        emailHeap.insert(email);

    }

/*

* NEXT displays the highest-priority email but does not
* remove it. Therefore, repeated NEXT commands display
* the same email until READ is executed.

*/

    void next()

```

```

{

    if (emailHeap.empty())

    {

        cout << "No unread emails." << endl;

        return;

    }

    emailHeap.getMax().display();

}

/*

* READ removes the current highest-priority email.

* It does not depend on NEXT being called first.

*/

void read()

{

    if (emailHeap.empty())

    {

        cout << "No unread emails." << endl;

        return;

    }

    emailHeap.removeMax();

}

void count() const

```

```

{
    cout << emailHeap.size() << endl;
}
};

/*
 * EmailProgram is responsible for reading and processing
 * commands from the test file.
 *
 * Expected EMAIL format:
 *
 * EMAIL sender,subject,date
 *
 * Example:
 * EMAIL Boss,Quarterly budget meeting,09-25-2026
 */
class EmailProgram
{
private:
    EmailPriorityQueue emailQueue;

    /*
     * Remove whitespace from the beginning and end of a string.
     */

```

```

string trim(const string& text) const
{
    size_t start = text.find_first_not_of(" \t\r\n");

    if (start == string::npos)

        return "";

    size_t end = text.find_last_not_of(" \t\r\n");

    return text.substr(start, end - start + 1);
}

/*
 * Process an EMAIL command.
 *
 * Subject lines may contain spaces, so commas are used
 * to separate the three fields.
 */

void processEmailCommand(const string& line)
{
    // Everything after the word EMAIL.

    string emailData = trim(line.substr(5));

    size_t firstComma = emailData.find(',');

    size_t secondComma = emailData.find(',', firstComma + 1);

    if (firstComma == string::npos ||

        secondComma == string::npos)

```

```

{
    cerr << "Invalid EMAIL command: " << line << endl;

    return;
}

string sender =

    trim(emailData.substr(0, firstComma));

string subject =

    trim(emailData.substr(firstComma + 1,

        secondComma - firstComma - 1));

string date =

    trim(emailData.substr(secondComma + 1));

Email newEmail(sender, subject, date);

emailQueue.addEmail(newEmail);
}

void processCommand(const string& line)

{
    string command = trim(line);

    if (command.empty())

        return;

    if (command.rfind("EMAIL", 0) == 0)

    {
        processEmailCommand(command);
    }
}

```

```

    }

    else if (command == "NEXT")
    {
        emailQueue.next();
    }

    else if (command == "READ")
    {
        emailQueue.read();
    }

    else if (command == "COUNT")
    {
        emailQueue.count();
    }

    else
    {
        cerr << "Unknown command: " << command << endl;
    }
}

public:

    /*
    * Read all commands from the specified test file.
    */

```

```

void run(const string& fileName)
{
    ifstream inputFile(fileName);

    if (!inputFile)
    {
        cerr << "Unable to open file: "
              << fileName << endl;

        return;
    }

    string line;

    while (getline(inputFile, line))
    {
        processCommand(line);
    }

    inputFile.close();
}

};

int main(int argc, char* argv[])
{
    /*
    * The input file can be supplied on the command line:
    *

```

```

* ./email_priority test.txt

*/

if (argc != 2)

{

    cerr << "Usage: " << argv[0]

        << " <test-file>" << endl;

    return 1;

}

EmailProgram program;

program.run(argv[1]);

return 0;

}

```

Gemini code:

```

#include
#include
#include
#include
#include
#include

//
=====
===
// Date Class: Handles parsing and comparison of MM-DD-YYYY dates
//
=====
===

```

```

class Date{
private:
    int month;
    int day;
    int year;

public:
    Date(int m = 1, int d = 1, int y = 2000) : month(m), day(d), year(y) {}

    // Parses a date string in MM-DD-YYYY format
    static Date parse(const std::string& dateStr) {
        int m = 1, d = 1, y = 2000;
        char dash1, dash2;
        std::stringstream ss(dateStr);
        if (ss >> m >> dash1 >> d >> dash2 >> y) {
            return Date(m, d, y);
        }
        return Date(1, 1, 2000); // Fallback default date
    }

    // Converts date to a comparable numeric format: YYYYMMDD
    int toNumeric() const {
        return year * 10000 + month * 100 + day;
    }

    bool operator<(const Date& other) const {
        return toNumeric() < other.toNumeric();
    }

    bool operator>(const Date& other) const {
        return toNumeric() > other.toNumeric();
    }

    bool operator==(const Date& other) const {
        return toNumeric() == other.toNumeric();
    }

    std::string toString() const {

```

```

        std::ostringstream ss;
        if (month < 10) ss << "0";
        ss << month << "-";
        if (day < 10) ss << "0";
        ss << day << "-" << year;
        return ss.str();
    }
};

//
=====

===
// Email Class: Represents an individual email and its priority rules
//
=====

===
class Email {
private:
    std::string senderCategory;
    std::string subject;
    Date date;
    long long arrivalOrder;
    static long long orderCounter;

    // Returns priority rank: higher number = higher priority
    int getCategoryPriority() const {
        if (senderCategory == "Boss") return 5;
        if (senderCategory == "Subordinate") return 4;
        if (senderCategory == "Peer") return 3;
        if (senderCategory == "ImportantPerson") return 2;
        if (senderCategory == "OtherPerson") return 1;
        return 0; // Unknown category lowest
    }

public:
    Email(const std::string& category, const std::string& subj, const Date& dt)
        : senderCategory(category), subject(subj), date(dt) {
        arrivalOrder = ++orderCounter;
    }
};

```

```

}

Email() : senderCategory("OtherPerson"), subject(""), date(Date()), arrivalOrder(0) {}

std::string getSenderCategory() const { return senderCategory; }
std::string getSubject() const { return subject; }
Date getDate() const { return date; }

// Defines priority ordering for MaxHeap
bool operator<(const Email& other) const {
    int thisPrio = getCategoryPriority();
    int otherPrio = other.getCategoryPriority();

    if (thisPrio != otherPrio) {
        return thisPrio < otherPrio;
    }

    // Tie-breaker: Newest email first (larger date = higher priority)
    if (!(date == other.getDate())) {
        return date < other.getDate();
    }

    // Secondary tie-breaker: FIFO arrival order
    return arrivalOrder < other.arrivalOrder;
}

bool operator>(const Email& other) const {
    return other < *this;
}
};

long long Email::orderCounter = 0;

//
=====
===
// MaxHeap Class: List-based MaxHeap priority queue implemented from scratch
//

```

=====

===

```
class MaxHeap {
private:
    std::vector heap; // List-based dynamic storage

    void heapifyUp(int index) {
        while (index > 0) {
            int parent = (index - 1) / 2;
            if (heap[parent] < heap[index]) {
                Email temp = heap[parent];
                heap[parent] = heap[index];
                heap[index] = temp;
                index = parent;
            } else {
                break;
            }
        }
    }

    void heapifyDown(int index) {
        int size = heap.size();
        while (true) {
            int leftChild = 2 * index + 1;
            int rightChild = 2 * index + 2;
            int largest = index;

            if (leftChild < size && heap[largest] < heap[leftChild]) {
                largest = leftChild;
            }
            if (rightChild < size && heap[largest] < heap[rightChild]) {
                largest = rightChild;
            }

            if (largest != index) {
                Email temp = heap[index];
                heap[index] = heap[largest];
                heap[largest] = temp;
            }
        }
    }
};
```

```

        index = largest;
    } else {
        break;
    }
}
}

```

public:

```

    MaxHeap() {}

```

```

    void insert(const Email& email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

```

```

    Email getMax() const {
        if (heap.empty()) {
            throw std::runtime_error("Heap is empty");
        }
        return heap[0];
    }

```

```

    void extractMax() {
        if (heap.empty()) {
            return;
        }
        heap[0] = heap.back();
        heap.pop_back();
        if (!heap.empty()) {
            heapifyDown(0);
        }
    }

```

```

    int size() const {
        return heap.size();
    }

```

```

    bool isEmpty() const {

```

```

        return heap.empty();
    }
};

//
=====
===
// CEOEmailManager Class: Parses commands and manages operations
//
=====
===
class CEOEmailManager {
private:
    MaxHeap emailQueue;

    std::string trim(const std::string& str) {
        size_t first = str.find_first_not_of(" \t\r\n");
        if (first == std::string::npos) return "";
        size_t last = str.find_last_not_of(" \t\r\n");
        return str.substr(first, (last - first + 1));
    }

public:
    void processCommand(const std::string& line) {
        std::string trimmedLine = trim(line);
        if (trimmedLine.empty() || trimmedLine[0] == '#') {
            return; // Ignore empty lines or comments
        }

        if (trimmedLine.rfind("EMAIL", 0) == 0) {
            std::string content = trim(trimmedLine.substr(5));
            size_t firstComma = content.find(',');
            size_t secondComma = content.find(',', firstComma + 1);

            if (firstComma != std::string::npos && secondComma != std::string::npos) {
                std::string category = trim(content.substr(0, firstComma));
                std::string subject = trim(content.substr(firstComma + 1, secondComma -
firstComma - 1));

```

```

        std::string dateStr = trim(content.substr(secondComma + 1));

        Date date = Date::parse(dateStr);
        Email email(category, subject, date);
        emailQueue.insert(email);
    }
}
else if (trimmedLine == "NEXT") {
    if (emailQueue.isEmpty()) {
        std::cout << "No unread emails.\n";
    } else {
        Email topEmail = emailQueue.getMax();
        std::cout << "Sender: " << topEmail.getSenderCategory() << "\n";
        std::cout << "Subject: " << topEmail.getSubject() << "\n";
        std::cout << "Date: " << topEmail.getDate().toString() << "\n";
    }
}
else if (trimmedLine == "READ") {
    if (emailQueue.isEmpty()) {
        std::cout << "No unread emails to read.\n";
    } else {
        Email topEmail = emailQueue.getMax();
        std::cout << "Sender: " << topEmail.getSenderCategory() << "\n";
        std::cout << "Subject: " << topEmail.getSubject() << "\n";
        std::cout << "Date: " << topEmail.getDate().toString() << "\n";
        emailQueue.extractMax();
    }
}
else if (trimmedLine == "COUNT") {
    std::cout << "Unread count: " << emailQueue.size() << "\n";
}
}

void runFromFile(const std::string& filename) {
    std::ifstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Error: Could not open file " << filename << "\n";
        return;
    }
}

```

```

    }

    std::string line;
    while (std::getline(file, line)) {
        processCommand(line);
    }
    file.close();
}
};

//
=====
===
// Main Entry Point
//
=====
===
int main(int argc, char* argv[]) {
    CEOEmailManager manager;

    std::string filename = "commands.txt";
    if (argc > 1) {
        filename = argv[1];
    }

    manager.runFromFile(filename);

    return 0;
}

```

e. Code analysis

- a. Correctness: Code A is more correct because it compiles and uses the required list-based MaxHeap. Code B has incomplete #include statements, uses a vector instead of the required list, and its READ command displays the email even though READ should remove it without displaying it
- b. Time complexity: Code A is less efficient because getIterator() must walk through the list every time it accesses a heap position. This makes insertion and removal much slower than the direct indexing Code B attempts to use

with a vector. However, B is not correct since the requirements ask for lists, not vectors.

- c. Space complexity: Both programs have $O(n)$ space complexity, but Code B would likely use less memory per email. Code A stores three strings and two integers for every email, and because it uses a `std::list`, every email also needs pointers connecting the list nodes. Code B stores only two strings, three integers for the date, and an `arrivalOrder`, while its intended vector does not require separate pointers for every element. With millions of emails, these little differences per email could add up to a significant amount of memory.
- d. Maintainability: Code A is easier to maintain because its classes are clearly separated and it follows the assignment's required structure more closely. Code B is also organized into classes and has useful comments, but the incorrect vector implementation, compilation errors, and extra `Date` and `arrival-order` logic add complexity.
- f. I selected the code generated by ChatGPT as the basis for my assignment. I chose the ChatGPT code because it follows the assignment requirements more closely than the Gemini code. In particular, ChatGPT uses a list to model the heap. The Gemini code uses a vector, which does not satisfy the assignment requirement for a list-based heap. The ChatGPT code also handles the `NEXT` and `READ` commands more accurately. `NEXT` is supposed to display the highest-priority email without removing it, while `READ` is supposed to remove the highest-priority email without displaying it. For these reasons, the ChatGPT code requires fewer corrections and provides a better starting point for the final program.
- g. I improved the ChatGPT code mainly for correctness and readability. For correctness, I changed the output's format so it better matches the assignment's sample output. For example, I updated `COUNT` so it prints a sentence such as "There are 6 emails to read" instead of only printing the number, and I updated `NEXT` so it labels the next email before displaying the sender, subject, and date. I also tested that `NEXT` displays the highest-priority email without removing it, and that `READ` removes the highest-priority email without displaying it. For execution time, I kept the `MaxHeap` approach from the ChatGPT code, but I did not replace the list with a vector because the assignment requires a list-based implementation. For space complexity, I did not add unnecessary extra variables, so the program still only stores the unread emails in the heap. For maintainability, I added clearer comments and kept the code organized into separate classes for the email, heap, priority queue, and program input handling.